

LLMs pour la Génération des Fonctions de Récompense : Réduire la Barrière entre l’Humain et l’Agent

Lyna Hocine, Mariana Duarte Ferreira

Novembre 2025

Résumé

En apprentissage par renforcement, la conception des fonctions de récompense efficaces est une tâche complexe et coûteuse, souvent susceptible à des erreurs et à des comportements inattendus. Les modèles de langage de grande taille (LLM) offrent de nouvelles pistes pour générer automatiquement ces fonctions à partir d’instructions en langage naturel, réduisant ainsi le besoin des données étiquetées ou de l’expertise dans le domaine.

Cet article présente une synthèse des différentes approches existantes : l’utilisation de prompts, la génération de feedbacks par les LLMs ou par les utilisateurs, et les méthodes hybrides combinant l’expertise humaine et les LLMs. Nous discutons également d’EUREKA [3], un système central dans ce domaine qui illustre l’approche intégrée de génération et d’optimisation de fonctions de récompense via les modèles de langage de grande taille. Enfin, nous analysons les limites actuelles de ces méthodes et les pistes proposées pour les dépasser.

Mots-clés : apprentissage par renforcement, LLM, fonctions de récompense, prompts, feedback

1 Introduction

Entraîner des agents par apprentissage par renforcement est une tâche difficile. Environ 92% des chercheurs dans le domaine interrogés rapportent que la conception manuelle des fonctions de récompense est basée sur des essais-erreurs, et 89% indiquent que ces récompenses sont sous-optimales et conduisent à des comportements inattendus [3].

Cette complexité est surtout due à la difficulté intrinsèque de concevoir une fonction de récompense efficace. Les méthodes traditionnelles reposent soit sur la conception manuelle d’une fonction de récompense, soit sur l’utilisation d’un grand volume de données étiquetées. Ces approches présentent plusieurs limites : elles ne se généralisent pas bien pour des utilisateurs qui ont des objectifs différents ou sans expertise dans le domaine. Toute modification des intentions de l’utilisateur nécessite une nouvelle conception de la fonction ou une nouvelle collection des données. Elles se montrent donc très coûteuses et peu modulables.

Les récents progrès des modèles de langage de grande taille (LLM) ont révolutionné la capacité à interpréter et agir sur des instructions en langage naturel. Leur utilisation pour la génération de fonctions de récompense a été largement explorée ces dernières années. Grâce à leur entraînement sur d’immenses ensembles de données textuelles, ces modèles présentent de fortes compétences en compréhension du langage naturel, en résolution de problèmes et en génération de contenu. Ils permettent ainsi de réduire la barrière entre les intentions exprimées par l’utilisateur et les actions de bas niveau de l’agent, et offrent de nouvelles possibilités pour rendre l’interaction plus efficace. Plusieurs approches ont émergé pour exploiter les LLMs dans ce contexte que ce soit pour interpréter l’objectif exprimé par l’utilisateur ou pour corriger la fonction au fil du temps.

Cet article explore donc l’interaction entre l’utilisateur et la machine, en posant la question centrale : comment l’utilisation des LLMs peut-elle rendre cette interaction plus naturelle et intuitive ?

Dans un premier temps, nous allons nous intéresser au premier échange entre l’utilisateur et le système, le prompt, avant de passer aux feedbacks nécessaires à l’affinement des fonctions de récompense. Nous arrivons ensuite sur l’étude d’EUREKA [3], un concepteur de fonctions de récompense à l’aide des LLM. Enfin, nous nous intéressons aux limites existantes actuellement sur ce sujet et les alternatives proposées.

2 Le prompt comme point d’entrée de l’interaction

La génération de fonctions de récompense commence toujours par un premier échange entre l’utilisateur et le système : le prompt. Les prompts sont utilisés pour interagir avec les LLMs afin d’accomplir une tâche. C’est une entrée donnée par l’utilisateur auquel le modèle doit répondre.

Les prompts peuvent inclure des instructions, des questions ou n’importe quel autre type d’entrée. Ils assurent le rôle de médiateur entre le langage humain, souvent peu clair, et la formalisation technique nécessaire pour l’apprentissage par renforcement. Les différents articles du corpus proposent plusieurs stratégies, allant du prompt totalement libre aux modèles plus structurés basés sur des templates.

2.1 Prompts libres fournis par l’utilisateur

Pour cette méthode, les prompts sont écrits entièrement par l’humain où l’utilisateur rédige une instruction en langage naturel décrivant l’objectif souhaité. Cette forme d’interaction est la plus intuitive puisqu’elle exploite la capacité de l’humain d’exprimer une intention avec ses propres mots. Les instructions peuvent ainsi être simples et directes. L’utilisateur peut spécifier son intention soit à l’aide d’une seule phrase quand les concepts sont bien connus, soit en spécifiant quelques exemples décrivant le comportement souhaité [2].

Cela rend le système très accessible : il permet à des utilisateurs non techniques de générer et d’orienter de nouveaux comportements robotiques sans exiger l’expertise pour concevoir des primitives de bas niveau. Les LLMs sont alors chargés d’interpréter ces descriptions pour dériver une fonction de récompense cohérente et opérationnelle.

Cependant, cette approche présente également plusieurs limites. Les descriptions en langage naturel sont souvent vagues, peu spécifiques et fortement dépendantes du contexte. Des termes simples comme “plus vite”, “plus fluide” ou “gracieusement” peuvent être difficiles à interpréter correctement pour un LLM et conduire à des comportements inattendus.

Les prompts libres représentent ainsi une première étape essentielle, mais relativement fragile, dans le processus de génération de fonctions de récompense.

2.2 Gabarits structurés à compléter

Une seconde approche proposée dans le corpus repose sur l’utilisation de templates structurés [7], c’est-à-dire des modèles prédéfinis que le système doit compléter à partir des prompts de l’utilisateur. Ces templates peuvent décrire un ensemble de mouvements courants pour un robot et sont utilisés comme base pour la génération de comportements. Le système utilisant cette approche peut également inclure un descripteur de mouvement dont le rôle principal est de prendre le prompt en langage naturel fourni par l’utilisateur et de l’intégrer dans le template en complétant certaines parties. Cette étape permet de transformer une instruction textuelle générale en une spécification de récompense plus formelle et mieux structurée.

L’avantage de cette technique est qu’elle permet d’extraire plus efficacement les connaissances internes des LLMs sur les mouvements robotiques. Elle réduit également l’ambiguïté et l’incohérence associées aux prompts libres. En imposant une structure fixe, le système parvient à produire des résultats plus cohérents et prévisibles. Contrairement à certains systèmes fondés sur les prompts libres, les utilisateurs n’ont pas besoin de fournir d’exemples ou de démonstrations pour guider le modèle : ils dépendent entièrement de la capacité du LLM à générer une fonction de récompense adéquate à partir du seul template.

Cependant, selon de nombreux articles du corpus[7], la principale limite de cette approche réside dans le fait que les templates doivent être conçus manuellement par des experts. Leur élaboration nécessite une connaissance approfondie du domaine ainsi qu’une anticipation exhaustive des variations de tâches que les utilisateurs pourraient souhaiter exprimer dans leurs prompts. De plus, la structure fixe des templates limite l’expressivité du système, ce qui peut réduire ses performances pour des tâches complexes ou très variées.

3 Le feedback comme second niveau d’échange

Après avoir généré une version de la fonction de récompense, le système s’appuie sur un retour, le feedback. Il joue un rôle essentiel pour corriger, affiner ou réévaluer la fonction de récompense à mesure que

l’agent explore son environnement. Les travaux du corpus mettent en évidence une diversité de stratégies : certains systèmes sollicitent directement le retour de l’utilisateur pour juger si le comportement produit correspond à l’objectif souhaité, tandis que d’autres délèguent cette tâche à des LLM capables de générer automatiquement du feedback.

3.1 Retour direct de l’utilisateur

Dans cette approche, le retour est fourni directement par l’utilisateur, qui évalue la qualité du comportement produit par le système au regard du comportement souhaité [7].

L’utilisateur peut comparer plusieurs résultats, indiquer lequel se rapproche le plus de l’objectif visé ou même apporter des corrections explicites à certaines actions. Cette forme de retour est représentée dans des méthodes de l’apprentissage par renforcement à travers de retours humains (RLHF), où l’expertise humaine sert à orienter l’apprentissage lorsque la fonction de récompense ne peut pas être formulée de manière précise ou exhaustive. Le principal avantage de cette méthode réside dans la capacité à mobiliser le jugement et l’intuition humaine, particulièrement efficaces pour saisir des nuances et des subtilités que les LLMs peinent encore à détecter.

Toutefois, cette approche présente des coûts importants : fournir un retour cohérent et continu constitue une tâche exigeante, qui sollicite fortement l’utilisateur et limite l’efficacité du processus. Un second inconvénient concerne la difficulté à assurer un retour fiable et scalable : maintenir une qualité de retour en temps réel devient rapidement complexe lorsqu’on doit évaluer un grand nombre de comportements ou des actions continues et variées.

3.2 Feedback généré automatiquement par les LLMs

Dans cette approche, le retour est fourni par le LLM lui-même. Une fois la fonction de récompense initiale générée et testée, les résultats obtenus sont présentés au modèle, accompagnés du taux de succès, d’une description du comportement observé ainsi que de l’objectif initial de la tâche. À partir de ces informations, le LLM identifie les limites de la fonction de récompense actuelle et produit un retour visant à corriger ou améliorer ces composantes.

Ce retour est ensuite réinjecté dans le modèle pour générer une version révisée de la fonction de récompense, créant ainsi une boucle d’auto-raffinement (self-refinement loop). Le LLM reçoit les données issues des entraînements, analyse les échecs et réussites, puis ajuste progressivement la fonction de récompense afin d’améliorer les performances du système [5][6].

Cette approche présente plusieurs avantages. Elle permet d’éliminer une partie des biais introduit par l’évaluation humaine, elle réduit fortement le coût associé à la collecte de feedback manuel et elle offre une solution hautement scalable. Cette capacité d’auto-évaluation rend le retour généré par les LLM puissant, notamment dans des environnements où les comportements doivent être évalués fréquemment et où la supervision humaine serait trop coûteuse ou trop lente. Elle présente également plusieurs limites. Le retour généré par les LLM reste susceptible d’hallucinations, d’erreurs d’interprétation ou de jugement incohérents. L’absence d’un évaluateur humain peut conduire à une dérive progressive de la fonction de récompense : sans supervision externe, le LLM peut converger vers des critères d’évaluation détachés de l’intention de l’utilisateur, notamment dans les tâches complexes ou mal définies.

3.3 Approches de feedback hybrides

Une approche hybride combine les avantages du feedback humain et du feedback généré par les LLMs [4]. Dans ce cadre, les LLMs participent à la fois à la construction de la fonction de récompense et à la détection des biais humains. L’idée centrale est que les LLMs peuvent analyser les retours humains, identifier des incohérences ou des biais, puis ajuster ou compléter la fonction de récompense en conséquence.

Comme certains problèmes restent trop complexes ou trop spécifiques pour être entièrement automatisés, cette approche repose sur un processus en deux étapes : le LLM génère la fonction de récompense et détecte d’éventuels biais dans les retours humains, ce qui permet aux humains de n’intervenir que dans des scénarios où leur expertise est nécessaire. L’intérêt de cette méthode est de réduire l’impact des biais humains, qui peuvent significativement dégrader les performances de l’agent si elles ne sont pas détectées et corrigées. Cette approche hybride permet donc de tirer parti de l’expertise humaine tout en utilisant les LLMs pour filtrer, analyser et corriger les biais. Elle garantit un meilleur équilibre entre l’intervention

humaine et la performance renforcée.

Elle présente cependant certaines limites. D’abord, obtenir un feedback de la part des LLMs nécessite des ressources computationnelles importantes, ce qui peut être coûteux et difficile à déployer dans certains contextes réels. De plus, cette méthode repose fortement sur une conception de prompts itérative et précise. Cela demande une expertise spécifique, car la qualité et la cohérence des retours produits par le modèle dépendent directement de la manière dont les prompts sont formulés.

4 EUREKA comme étude centrale

Dans cette partie, nous allons approfondir sur l’algorithme utilisé par EUREKA (**E**volution-driven **U**niversal **R**eward **K**it for **A**gent) [3], un concepteur de fonction de récompense au niveau humain alimenté par des LLMs. Celle-ci exploite une génération sans utiliser des exemples (*zero-shot* en anglais), de l’écriture de code et des capacités d’amélioration en contexte.

4.1 Algorithme employé par EUREKA

EUREKA a trois composantes algorithmiques principales : 1) l’environnement dans le contexte, 2) la recherche par algorithmes évolutionnaires et enfin, 3) la réflexion sur la récompense.

4.1.1 Environnement dans le contexte

Lors du premier prompt, EUREKA fournit directement une description de la tâche en langage naturel ainsi que le code source de l’environnement. Lorsqu’une fonction de récompense existe déjà, celle-ci n’est pas incluse. L’idée principale est que le modèle sache quelles sont les variables d’état et d’action, étant donné que les fonctions de récompense se basent toujours sur ces variables. Dans les cas où le code source n’est pas disponible, les informations nécessaires seront fournies depuis l’utilisation d’une API, par exemple.

Une fois que le code de l’environnement est fourni, **EUREKA demande explicitement au LLM de générer une fonction de récompense sous la forme d’un code Python exécutable.**

Ce prompt contient également des conseils génériques sur la conception de la récompense et sur la mise en forme. Avec seulement ces instructions, au premier essai, EUREKA peut déjà générer des fonctions de récompense plausibles pour différents environnements. Il est cependant important de noter que le code n’est pas toujours exécutable lors de la première itération.

À titre illustratif, le premier prompt fourni au LLM est structuré de la manière suivante :

- Le prompt commence par instruire le LLM à raisonner comme un ingénieur chargé de créer des fonctions de récompenses efficaces pour des tâches d’apprentissage par renforcement.
- Il explique ensuite au LLM à quoi la fonction de récompense retournée doit ressembler, quelles variables de l’environnement doivent être utilisées comme entrées, et avec quels packages (par exemple PyTorch) la fonction doit être compatible.
- Des exemples de signature de fonction sont fournis.
- Enfin, le LLM est invité à générer directement une fonction de récompense exécutable, sans inclure de fonction de récompense préexistante.

4.1.2 Recherche par algorithmes évolutionnaires

La recherche par algorithmes évolutionnaires permet d’adresser les erreurs d’exécution et les défis de sous-optimalité.

Dans chaque itération, EUREKA échantillonne plusieurs sorties indépendantes du LLM. **Autrement dit, le même prompt est fourni en entrée K fois au LLM et les fonctions de récompenses générées sont ensuite enregistrées.** Comme les générations sont i.i.d (indépendantes et identiquement distribuées), la probabilité que toutes les fonctions retournées contiennent des erreurs décroît exponentiellement selon la taille de l’échantillon. L’étude d’EUREKA conclut, pour les environnement choisis, que pour un échantillon de taille 16, au moins une fonction de récompense sera exécutable à la première itération.

Cet échantillonnage permet ainsi d’assurer la diversité des fonctions de récompense candidates et la solidité du processus, en augmentant la probabilité d’obtenir rapidement des fonctions exploitables pour la suite.

Après avoir fourni une première fonction exécutable, EUREKA fait une mutation de la récompense en proposant une amélioration par rapport à la meilleure fonction générée à l'itération précédente.

Dans ce contexte, une mutation désigne une modification guidée de la fonction de récompense, consistant à réécrire, ajuster ou ajouter des composantes de la récompense à partir du retour fourni par la réflexion sur la récompense dans le but d'améliorer la performance de la fonction.

Concrètement, à l'itération $t + 1$, EUREKA va prendre la fonction la plus performante de l'itération t , la réflexion sur la récompense (Section 4.1.2) et le prompt de mutation comme contexte et génère K autres sorties i.i.d par le LLM. Cette optimisation itérative continue jusqu'à ce qu'un nombre spécifié d'itérations soit atteint.

À titre d'exemple, le prompt de la mutation est construit comme suit :

- On explique qu'un agent a été entraîné en apprentissage par renforcement avec la fonction de récompense et que les valeurs des composantes et des métriques globales (taux de succès, longueur des épisodes, etc.) sont fournies au LLM à l'aide de la réflexion sur la récompense (Section 4.1.2).
- Le LLM doit analyser ce feedback et proposer une fonction de récompense optimisée, en suivant ces recommandations :
 - Réécrire la fonction si les taux de succès restent proches de zéro.
 - Ajuster, réécrire ou supprimer toute composante dont la valeur reste quasi constante.
 - Redimensionner toute composante dont la magnitude est excessive par rapport aux autres.
- Enfin, le LLM applique cette analyse pour chaque composante et écrit le code de la nouvelle fonction de récompense.

4.1.2 Réflexion sur la récompense

Dans EUREKA, la *réflexion sur la récompense* désigne un mécanisme de retour automatique, exprimé sous forme textuelle, qui analyse la qualité et l'évolution des différentes composantes de la fonction de récompense au cours de l'entraînement.

Afin de fonder la mutation en contexte de la récompense, la qualité des récompenses générées doit être exprimée par des mots. C'est pourquoi, dans ce cadre, la réflexion de récompense, un retour automatique qui résume les dynamiques des politiques d'entraînement sous forme de texte, est introduite. Au cours de la politique d'entraînement, EUREKA évalue individuellement toutes les composantes de la récompense à plusieurs moments différents de l'entraînement. Cela permet de ne pas avoir seulement un score final qui n'apporte pas beaucoup d'information mais plutôt une compréhension précise de ce qui fonctionne ou non dans la récompense et pour quelles raisons.

Cette procédure est importante pour deux raisons : (1) l'absence d'un signal d'amélioration suffisamment fin dans la fonction de performance de la tâche et (2) la dépendance de l'optimisation de la récompense. Autrement dit, elle évite un feedback vague et imprécis, en fournissant un retour adapté à l'algorithme utilisé lors de la phase d'entraînement. Ainsi, EUREKA génère un compte rendu qui détaille la performance de l'algorithme d'apprentissage pour chaque composante de la récompense de manière ciblée.

Pour clarifier, le processus de réflexion utilisé par EUREKA pour améliorer les fonctions de récompense se déroule de la manière suivante :

- Après chaque itération, EUREKA entraîne une politique d'apprentissage par renforcement avec la fonction de récompense générée et collecte des métriques de performance globales (par exemple le taux de réussite) ainsi que les valeurs individuelles de chaque composante de la récompense.
- Ces informations sont synthétisées dans un retour textuel, appelé *réflexion de récompense*, qui décrit les forces et faiblesses de chaque composante. Par exemple, si une composante reste constante malgré l'entraînement, cela indique qu'elle n'est pas optimisable dans sa forme actuelle.
- Sur la base de cette réflexion, le LLM est guidé pour générer de nouvelles fonctions de récompense en modifiant ou ajoutant des composantes selon le retour.

4.2 Mise en perspective

Dans EUREKA, l'utilisateur exprime ses besoins en langage naturel à travers un prompt sans avoir besoin de fournir d'exemples. Le prompt est essentiel pour la génération de la fonction de récompense car il permet au LLM de savoir quelles sont les attentes sur le comportement de l'agent.

De plus, deux approches principales de feedback sont explorées pour l’algorithme d’EUREKA. La première est entièrement basée sur le modèle de langage : le modèle génère une réflexion sur la récompense pour chaque fonction générée, ce qui sert à guider l’itération suivante. Même si cette méthode reste centrale dans leurs expériences, les auteurs explorent aussi la possibilité d’intégrer le retour humain, ce qui est particulièrement utilisé pour des tâches plus complexes ou lorsque l’agent n’atteint pas le comportement désiré. Suite à leurs expériences, ils concluent que la majorité des utilisateurs testant les politiques entraînées préfèrent celles produites avec le retour humain. Ils soulignent que cette approche conduit à un comportement plus sûr et plus aligné avec les préférences humaines.

Ces résultats sont particulièrement intéressants, car on pourrait penser que les utilisateurs ne souhaitent pas avoir à fournir un effort supplémentaire. Cependant, EUREKA montre que, malgré les capacités des LLM à imiter le comportement humain, ils ne sont pas encore capables de produire des récompenses entièrement fiables de manière autonome.

4.3 Approche expérimentale et évaluation des performances

EUREKA a été testé sur 10 robots différents et 29 tâches. Il a été comparé avec trois références de base :

- Des fonctions de récompense conçues par des experts dans le domaine.
- Des indicateurs binaires simples (*sparse rewards*).
- L2R [7], qui se repose notamment sur des templates fixes et prédéfinis.

Afin d’assurer une juste comparaison, tous les tests ont été effectués avec le même algorithme d’apprentissage par renforcement.

Les expérimentations ont montré qu’EUREKA égalise ou surpasse les performances humaines dans la grande majorité des tâches. En revanche, L2R [7] donne des résultats satisfaisants pour les tâches plus simples, mais se montre moins performant sur celles plus complexes, ce qui met en avant les faiblesses de son approche par modèles.

Des tests supplémentaires ont montré que l’utilisation d’une boucle de raffinement autonome améliore considérablement la qualité des fonctions de récompense générées par EUREKA.

Ces résultats sont particulièrement intéressants car ils confirment l’efficacité d’EUREKA à s’adapter et à optimiser ses fonctions de récompense de manière autonome, renforçant ainsi sa performance sur des tâches variées.

5 Limites actuelles et alternatives explorées

5.1 Limites liées à la description textuelle

Actuellement, un utilisateur peut exprimer ses besoins en langage naturel, en fournissant ou pas des exemples. Cette approche permet à la description de plusieurs scénarios possibles. Cependant, il peut être très compliqué de décrire certains comportements en utilisant seulement le langage naturel. Par exemple, comment expliquer ce qui est un négociateur “polyvalent” ? Comment faire comprendre à un robot humanoïde les mouvements nécessaires pour “courir vite” ?

Les limites sont donc évidentes. Nous ne sommes pas toujours capables de décrire en langage naturel le comportement que l’on souhaite faire apprendre à l’agent. Cette possible incohérence dans la description peut porter à des récompenses inefficaces ou à des comportements imprécis.

Une solution envisageable consiste à utiliser des vidéos. En fournissant des exemples visuels du comportement souhaité, nous pourrions réduire une grande partie de cette imprécision. Des travaux [8] explorent déjà cette approche, mais elle reste pour l’instant expérimentale.

5.2 Limites liées à la taille des LLMs

Actuellement, les articles de corpus s’appuient souvent de très grands modèles de langage comme GPT-4o en partant du principe que la taille importante de leurs bases de connaissances leur permet une plus grande diversité, ce qui réduit la probabilité de rencontrer des situations inédites lors du déploiement du robot entraîné à l’aide de ces modèles.

Une solution proposée pour dépasser cette limitation consiste à utiliser des LLMs de petite taille et à les entraîner à travers leur interaction directe avec l’environnement, plutôt que de dépendre uniquement de leurs bases de données.

Les expériences menées dans ce cadre [1] montrent que le déploiement embarqué de petits modèles est non seulement possible, mais qu’il surpasse les performances de modèles plus larges lorsqu’ils sont entraînés via des retours issus de l’environnement. De plus, cette approche répond à des contraintes liées au coût, à la consommation énergétique et à la latence, des problèmes auxquels les très grands modèles font face.

Cependant, cette approche reste expérimentale et présente plusieurs limites, notamment une faible capacité de passage à l’échelle.

6 Conclusion

La génération des fonctions de récompense est une tâche très complexe qui peut être résolue par plusieurs approches. Traditionnellement, cette génération reposait sur des méthodes manuelles ou l’exploitation de grandes quantités de données étiquetées, souvent coûteuses et peu flexibles. Récemment, une nouvelle méthode a émergé : une génération à l’aide des modèles de langage de grande taille, qui se montrent très efficaces et précis. Ces modèles offrent la possibilité d’interpréter des instructions en langage naturel, d’analyser et d’affiner automatiquement les fonctions de récompense, tout en réduisant la dépendance à l’expertise humaine.

Dans cette étude, nous nous sommes intéressés à la manière dont l’utilisation de ces modèles peut rendre l’interaction entre l’utilisateur et l’agent plus naturelle et intuitive. Nous avons d’abord analysé le rôle du prompt comme première interface, montrant que les LLMs facilitent l’expression des besoins en langage courant et simplifient ainsi la génération de fonctions de récompense.

Nous avons ensuite étudié les différentes manières de fournir du feedback sur les fonctions générées : retours complets, feedbacks produits par les LLMs ou méthodes hybrides. Les modèles se révèlent particulièrement efficaces pour produire des évaluations rapides, repérer des incohérences et compléter les retours humains. L’analyse de l’algorithme d’EUREKA [3] illustre la combinaison de ces approches : prompts, feedbacks automatiques et boucles d’auto-raffinement, montrant comment les LLMs peuvent automatiser plusieurs niveaux d’interaction.

Enfin, nous avons mis en évidence quelques limites actuelles, telles que la difficulté à formuler certaines tâches complexes en langage naturel ou encore les coûts importants liés aux grands modèles. Nous avons également évoqué diverses pistes pour y remédier, notamment l’usage de modèles plus compacts ou l’intégration de données multimodales. Malgré ces contraintes, ces approches ouvrent la voie à des systèmes toujours plus fiables, efficaces et accessibles.

En conclusion, les modèles de langage de grande taille occupent une place déterminante dans l’amélioration de la fluidité de l’interaction humain-machine dans le cadre de la génération de fonctions de récompense. Ils permettent de traduire les intentions des utilisateurs en objectifs clairs pour les agents.

L’intégration des LLMs ouvre donc des perspectives prometteuses pour rendre la génération des fonctions de récompense et par extension l’apprentissage par renforcement plus efficace, adaptable et proche des besoins des utilisateurs.

Références

- [1] Liam Boyle, Nicolas Baumann, Paviththiren Sivasothilingam, Michele Magno, and Luca Benini. Robotxr1 : Enabling embodied robotic intelligence on large language models through closed-loop reinforcement learning. *arXiv preprint*, 2025.
- [2] Minae Kwon, Sang Michael Xie, Kalesha Bullard, and Dorsa Sadigh. Reward design with language models. In *International Conference on Learning Representations*, 2023.
- [3] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka : Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Mohammad Saif Nazir and Chayan Banerjee. Zero-shot llms in human-in-the-loop rl : Replacing human feedback for reward shaping. *arXiv preprint*, 2025.
- [5] Jiayang Song, Zhehua Zhou, Jiawei Liu, Chunrong Fang, Zhan Shu, and Lei Ma. Self-refined large language model as automated reward function designer for deep reinforcement learning in robotics. *arXiv preprint*, arXiv :2309.06687, 2023.
- [6] Guanwen Xie, Jingzehua Xu, Yiyuan Yang, Yimian Ding, and Shuai Zhang. Large language models as efficient reward function searchers for custom-environment multi-objective reinforcement learning. *arXiv preprint*, 2024.
- [7] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. *Proceedings of the Conference on Robot Learning*, 229 :374–404, 2023.
- [8] Runhao Zeng, Dingjie Zhou, Qiwei Liang, Junlin Liu, Hui Li, Changxin Huang, Jianqiang Li, Xiping Hu, and Fuchun Sun. Video2reward : Generating reward function from videos for legged robot behavior learning. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI 2024)*, volume 392 of *Frontiers in Artificial Intelligence and Applications*, pages 4369–4376. IOS Press, 2024.

A Méthodologie et sélection de la littérature

Le domaine de génération de fonctions de récompense pour l'apprentissage par renforcement, guidé par les modèles de langage de grande taille, est encore récent et évolue rapidement. De nombreuses contributions dans ce domaine sont d'abord publiées sous forme de preprints, notamment via arXiv, avant leur publication.

Afin de refléter l'état de l'art le plus actuel, ces travaux ont été inclus dans notre corpus, à l'addition des articles publiés dans des conférences reconnues. Leur sélection repose sur les critères suivants :

1. La pertinence par rapport aux limitations identifiées du domaine, telles que la dépendance aux très grands LLMs [1], les biais dans le feedback humain [4] ou la difficulté à décrire certains comportements uniquement en langage naturel [8].
2. La présence de mécanismes méthodologiques innovants ou de paradigmes d'amélioration autonome [5][6].
3. La solidité expérimentale et la validité des résultats présentés [2][3][7].
4. La contribution à des applications concrètes, en particulier dans le domaine de l'apprentissage robotique [3][7].